Visual Explanation of Recursive Tribonacci

A clear, modern walkthrough of the code, its logic, pitfalls, edge cases, and complexity analysis.

The Code

```python
def tribonacci(n: int) -> int:
    """Calculate the nth tribonacci number using recursion."""
    if n == 0 or n == 1:
        return n
    if n == 2:
        return 1
    return tribonacci(n - 1) + tribonacci(n - 2) + tribonacci(n - 3)

tribonacci(5)
```

1. What Problem Does This Code Solve?

The function calculates the **nth Tribonacci number**. Tribonacci is similar to Fibonacci, but instead of adding the previous **two** numbers, it adds the previous **three** numbers.

Definition

$$T(0) = 0, \qquad T(1) = 1, \qquad T(2) = 1$$
$$T(n) = T(n-1) + T(n-2) + T(n-3) \quad \text{for } n \geq 3$$

The beginning of the sequence is:

$$0,\ 1,\ 1,\ 2,\ 4,\ 7,\ 13,\ 24,\ 44, \ldots$$

So the final call `tribonacci(5)` returns **7**.

2. Line-by-Line Explanation

Code	Meaning
def tribonacci(n: int) −> int:	Defines a function that accepts an integer `n` and returns an integer.
if n == 0 or n == 1: return n	Handles two base cases: `T(0)=0` and `T(1)=1`.
if n == 2: return 1	Handles the third base case: `T(2)=1`.
return tribonacci(n − 1) + ...	Solves the problem recursively by reducing it into three smaller Tribonacci problems.

3. The Core Logic

The function follows a classic recursive pattern:

Step 1: Check whether the input is small enough to answer immediately.

Step 2: If not, break the problem into smaller versions of itself.

Step 3: Add the answers from those smaller problems.

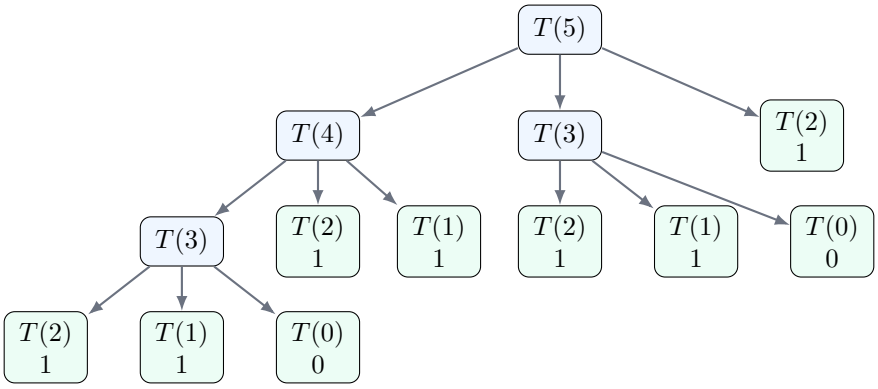
Step 4: Keep doing this until every branch reaches a base case.

> **Mental Model**
>
> To compute `tribonacci(n)`, ask: "What are the three Tribonacci numbers immediately before `n`?" Then add them.

For example:

$$T(5) = T(4) + T(3) + T(2) = 4 + 2 + 1 = 7$$

4. Visual Recursion Tree for `tribonacci(5)`



Green nodes are **base cases**; they stop the recursion and return immediately. Blue nodes are **recursive cases**; they still need to split into smaller calls.

5. Manual Calculation of `tribonacci(5)`

$$T(0) = 0, \quad T(1) = 1, \quad T(2) = 1$$
$$T(3) = T(2) + T(1) + T(0) = 1 + 1 + 0 = 2$$
$$T(4) = T(3) + T(2) + T(1) = 2 + 1 + 1 = 4$$
$$T(5) = T(4) + T(3) + T(2) = 4 + 2 + 1 = 7$$

Therefore:

$$\boxed{\texttt{tribonacci(5)} = 7}$$

6. How to Approach This Type of Problem

Recursive Problem-Solving Checklist

1. **Identify the smallest inputs.** Here they are `0`, `1`, and `2`.
2. **Write base cases first.** These prevent infinite recursion.
3. **Find the recurrence relation.** Here, each value depends on the previous three values.
4. **Ensure each recursive call moves toward a base case.** The calls use `n-1`, `n-2`, and `n-3`.
5. **Trace a small example.** Use `n=3`, `n=4`, or `n=5` before trusting the function.

7. Common Pitfalls

- **Missing base cases:** Without `n == 0`, `n == 1`, and `n == 2`, recursion may continue forever or produce incorrect results.
- **Incorrect recurrence:** Tribonacci uses three previous values, not two like Fibonacci.
- **Negative inputs:** This implementation does not handle negative `n`; `tribonacci(-1)` would recurse endlessly until Python raises a recursion error.
- **Repeated work:** The same values, such as `T(3)` and `T(2)`, are recomputed many times.
- **Large inputs:** Plain recursion becomes very slow because the call tree grows exponentially.

8. Edge Cases

Input	Expected Result	Reason
`tribonacci(0)`	`0`	First base case.
`tribonacci(1)`	`1`	Second base case.
`tribonacci(2)`	`1`	Third base case.
`tribonacci(3)`	`2`	First true recursive calculation.
`tribonacci(5)`	`7`	Combines `T(4)`, `T(3)`, and `T(2)`.
`tribonacci(-1)`	Problematic	No negative-input guard exists.

A safer production version would reject negative inputs:

```python
def tribonacci(n: int) -> int:
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0 or n == 1:
        return n
    if n == 2:
        return 1
    return tribonacci(n - 1) + tribonacci(n - 2) + tribonacci(n - 3)
```

9. Time Complexity

Each non-base call creates **three more calls**:

$$T(n) = T(n-1) + T(n-2) + T(n-3) + O(1)$$

The exact growth is slightly less than 3^n, but the key idea is that the number of calls grows **exponentially**. A simple upper-bound trick is:

$$T(n) \leq 3T(n-1) + O(1)$$

So:

$$\boxed{\text{Time Complexity} = O(3^n)}$$

Easy Complexity Trick

If a recursive function makes a calls and each call reduces the input by about 1, estimate the time as $O(a^n)$. Here, there are three recursive calls, so the quick estimate is $O(3^n)$.

10. Space Complexity

Space complexity comes mainly from the **recursion call stack**. Even though the recursion tree has many total calls, Python does not keep the entire tree active at once. It follows one path down, then returns.

The longest path is:

$$n \to n-1 \to n-2 \to \cdots \to 0$$

That path has length proportional to n, so:

$$\boxed{\text{Space Complexity} = O(n)}$$

Easy Space Trick

For recursive functions, ask: "What is the maximum depth of the call stack?" If the input decreases by 1 each level, the stack depth is usually $O(n)$.

11. How to Improve the Code

The main weakness of the original version is repeated computation. For example, `T(3)` and `T(2)` are calculated multiple times. This can be fixed with **memoization**, which stores results after computing them once.

```python
from functools import lru_cache

@lru_cache(None)
def tribonacci(n: int) -> int:
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0 or n == 1:
        return n
    if n == 2:
        return 1
    return tribonacci(n - 1) + tribonacci(n - 2) + tribonacci(n - 3)
```

With memoization:

- Each value from `0` to `n` is computed once.
- Time complexity improves from $O(3^n)$ to $O(n)$.
- Space complexity remains $O(n)$ because cached values and recursion stack both grow with `n`.

12. Final Summary

Key Takeaways

- Tribonacci adds the previous **three** values: $T(n-1) + T(n-2) + T(n-3)$.
- The base cases are $T(0) = 0$, $T(1) = 1$, and $T(2) = 1$.
- `tribonacci(5)` returns **7**.
- Plain recursion is elegant but inefficient because it repeats work.
- Time complexity is $O(3^n)$; space complexity is $O(n)$.
- Memoization improves time complexity to $O(n)$.